

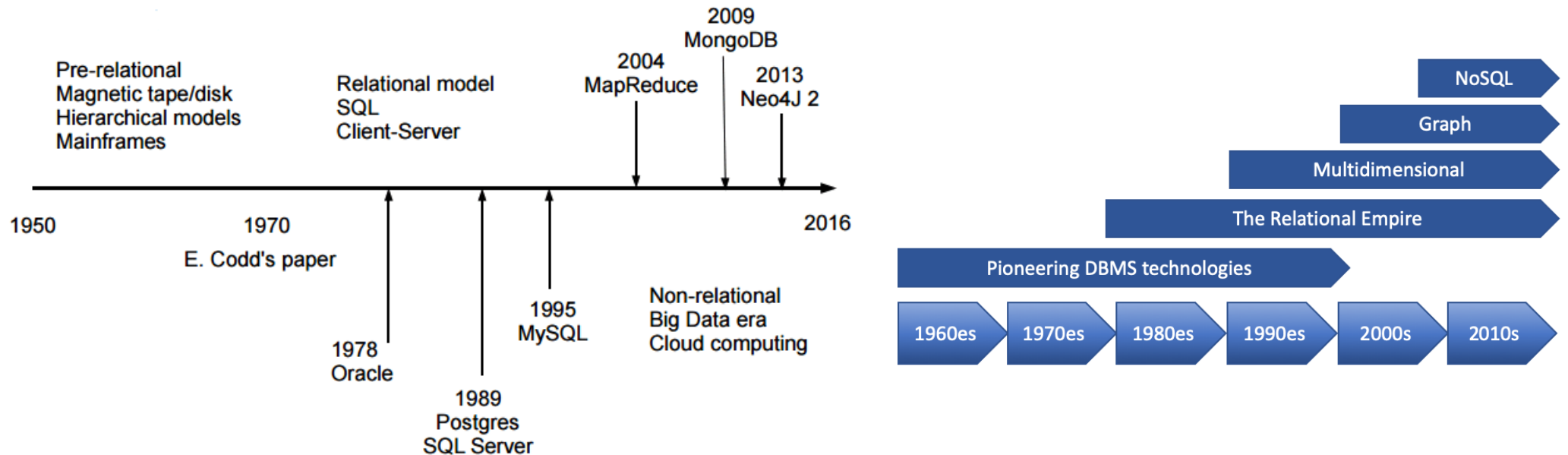
Database Management System

View, Stored Procedures and Functions P2

Dr. Tran Anh Tuan,

The database evolution happened in five “waves”:

- The first wave consisted of network, hierarchical, inverted list, and (in the 1990's) object-oriented DBMSs; it took place from roughly 1960 to 1999.
- The relational wave introduced all of the SQL products (and a few non-SQL) around 1990 and began to lose users around 2008.
- The decision support wave introduced Online Analytical Processing (OLAP) and specialized DBMSs around 1990, and is still in full force today.
- The graph wave began with The Semantic Web stack from the Worldwide Web Consortium in 1999, with property graphs appearing around 2008
- The NoSQL wave includes big data and much more; it began in 2008.



Định nghĩa Hàm $\neq$ view

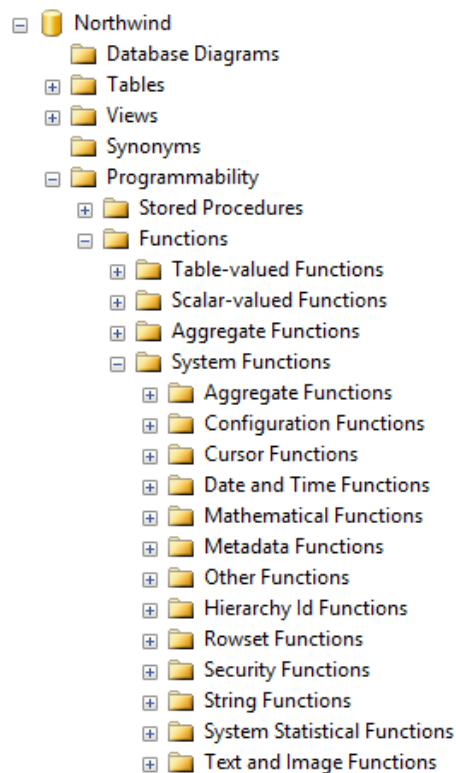


Hàm là một đối tượng trong cơ sở dữ liệu bao gồm một tập nhiều câu lệnh SQL được nhóm lại với nhau thành một nhóm. Điểm khác biệt là hàm trả về một giá trị thông qua tên hàm. Điều này cho phép ta sử dụng hàm như là một thành phần của một biểu thức chẳng hạn như trong các câu lệnh truy vấn

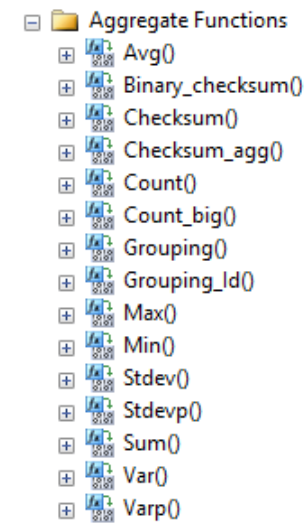
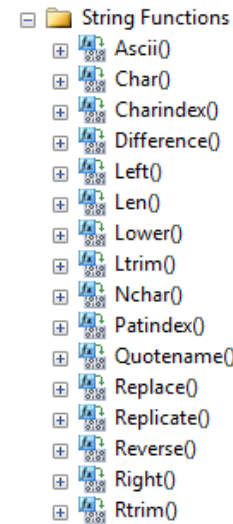
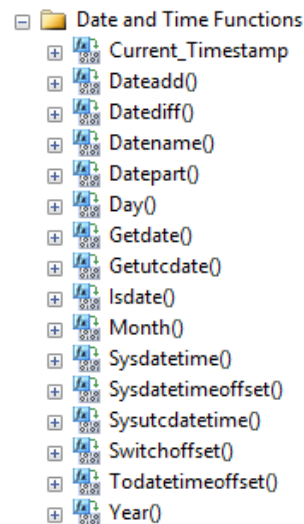
- Trong SQL có rất nhiều các hàm được định nghĩa sẵn (Được chia theo nhóm – Trong 1 Database bạn chọn Programmability/Functions/System Functions) như các hàm về chuỗi (String Functions), các hàm về ngày tháng (Date and Time Functions), Các hàm toán học (Mathematical Function), ...

Code	thứ mấy	flag	Copy
<pre>IF DATENAME(<u>weekday</u>, GETDATE()) IN (N'Saturday', N'Sunday') SELECT 'Weekend'; ELSE SELECT 'Weekday';</pre>			

Định nghĩa Hàm



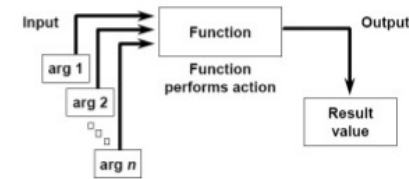
- Ngoài những hàm do hệ quản trị cơ sở dữ liệu cung cấp sẵn, bạn có thể tự xây dựng các hàm nhằm phục vụ cho mục đích riêng của mình
- Các hàm do người dùng định nghĩa. Các hàm do người dùng định nghĩa thường có 2 loại: Loại 1 là Hàm với giá trị trả về là “**dữ liệu kiểu bảng**” – **Table-valued Functions**; Loại 2 là Hàm với giá trị trả về là **một giá trị** – **Scalar-valued Functions** trả về 1 giá trị / 1 number



Cú pháp Hàm

hàm truyền tham số động - *a (gần giống con trỏ/ tham chiếu)

SQL Functions



```
CREATE FUNCTION Ten_Ham ( [Danh_Sach_Cac_Tham_So] )  
RETURNS Kieu_Du_Lieu_Tra_Ve_Cua_Ham  
AS  
BEGIN  
    Cac_Cau_Lenh_Cua_Ham  
END
```

- – Ten_Ham: Tên của hàm cần tạo.
- – Danh_Sach_Cac_Tham_So: Các tham số của hàm được khai báo ngay sau tên hàm và được bao bởi cặp dấu (), Danh sách các tham số này có thể không có
- – Cac_Cau_Lenh_Cua_Ham: Tập hợp các câu lệnh sử dụng trong nội dung hàm để thực hiện các yêu cầu của hàm.

Ví dụ Hàm



- Hàm trả về giá trị là năm hiện hành (Theo giờ hệ thống trên máy Database server):

```
CREATE FUNCTION dbo.fuGetCurrYear()  
RETURNS int  
AS  
BEGIN  
    RETURN YEAR(getdate())  
END
```

tiết độ ngữ (pointing to the function name)

```
SELECT dbo.fuGetCurrYear() AS 'Current Year'
```

Results		Messages
	Current Year	
1	2017	

```
SELECT * FROM [Order]  
WHERE YEAR(OrderDate) <= dbo.fuGetCurrYear ()
```

Results		Messages			
	Id	OrderDate	OrderNumber	CustomerId	TotalAmount
1	1	2012-07-04 00:00:00.000	542378	85	10.00
2	2	2012-07-05 00:00:00.000	542379	79	1863.40
3	3	2012-07-08 00:00:00.000	542380	34	1813.00

Ví dụ Hàm



- Hàm trả về số ngày của tháng, năm do bạn truyền vào

```
CREATE FUNCTION dbo.fuDaysInMonth (@Thang Int, @Nam Int)
RETURNS int
AS
BEGIN
    DECLARE @Ngay Int    khai báo biến
    IF @Thang = 2
    BEGIN
        IF ((@Nam % 4 = 0 AND @Nam % 100 <> 0) OR (@Nam % 400 = 0))
            SET @Ngay = 29
        ELSE
            SET @Ngay = 28
        END
    ELSE
        SELECT @Ngay = CASE WHEN @Thang IN (1,3,5,7,8,10,12) THEN 31 ELSE 30 END
    RETURN @Ngay
END
```

truyền vào ?
giá trị trả về ?
chức năng cụ thể ?

```
SELECT dbo.fuDaysInMonth(2,2017) AS NumberOfDay
```

Results		Messages	
NumberOfDay			
1	28		

Ví dụ Hàm



- Hàm xác định thứ trong tuần của một giá trị kiểu ngày

```
CREATE FUNCTION fuThu(@ngay DATETIME)
RETURNS NVARCHAR(10)
AS
BEGIN
    DECLARE @KetQua NVARCHAR(10)
    SELECT @KetQua=CASE DATEPART(DW,@ngay)
        WHEN 1 THEN N'Chủ nhật'
        WHEN 2 THEN N'Thứ hai'
        WHEN 3 THEN N'Thứ ba'
        WHEN 4 THEN N'Thứ tư'
        WHEN 5 THEN N'Thứ năm'
        WHEN 6 THEN N'Thứ sáu'
        ELSE N'Thứ bảy'
    END
    RETURN (@KetQua) /* Trị trả về của hàm */
END
```

trả về số, còn datename trả về name

unicode

```
SELECT dbo.fuThu('2017-04-14') AS N'Thứ Trong Ngày'
```

100 %	
Results	Messages
Thứ Trong Ngày	
1	Thứ sáu

Ví dụ Hàm



- Let us see another example which will fetch the number of orders processed by an employee for a given year. Write the following function in our query pad

```
CREATE FUNCTION FetchEmployeeProcessedOrdersYearWise
(
    @p_EmployeeID INT,
    @p_Year INT
) RETURNS INT
BEGIN
    RETURN (SELECT COUNT(OrderID) FROM Orders
    WHERE EmployeeID=@p_EmployeeID AND YEAR(OrderDate)=@p_Year)
END

--Test the function
SELECT dbo.FetchEmployeeProcessedOrdersYearWise(1,1996) AS 'Year 1996'
SELECT dbo.FetchEmployeeProcessedOrdersYearWise(1,1997) AS 'Year 1997'
SELECT dbo.FetchEmployeeProcessedOrdersYearWise(1,1998) AS 'Year 1998'
```

DEFAULT trong Function



- Ta có thể gán giá trị mặc định cho tham số truyền vào trong nhiều trường hợp.

```
CREATE FUNCTION CalculateNumber(@Num1 int = 0, @Num2 int = 0 )  
RETURNS int  
AS  
BEGIN  
    DECLARE @Result INT  
    SET @Result = @num1 + @num2  
    RETURN @Result  
END
```

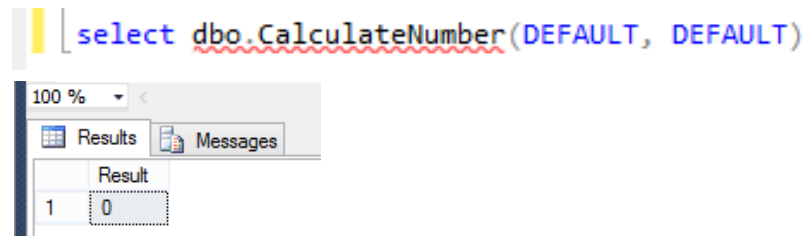


Table-value Functions



- Ta đã biết được chức năng cũng như sự tiện lợi của việc sử dụng các khung nhìn (View) trong cơ sở dữ liệu. Tuy nhiên, nếu cần phải sử dụng các tham số trong khung nhìn (chẳng hạn các tham số trong mệnh đề WHERE của câu lệnh SELECT) thì ta lại không thể thực hiện được. Điều này phần nào đó làm giảm tính linh hoạt trong việc sử dụng khung nhìn.

```
CREATE VIEW sinhvien_k25
AS
SELECT masv,hodem,ten,ngaysinh
FROM sinhvien INNER JOIN lop
ON sinhvien.malop=lop.malop
WHERE khoa=25
```

```
SELECT * FROM sinhvien_K25|
```

*Ta có thể biết được danh sách các sinh viên khoá 25 một cách dễ dàng nhưng rõ ràng không thể thông qua khung nhìn này để biết được danh sách sinh viên các khoá khác do không thể sử dụng điều kiện có dạng **KHOA = @thamso***

Table-value Functions



- Nhược điểm trên của khung nhìn có thể khắc phục bằng cách sử dụng hàm với giá trị trả về dưới dạng bảng và được gọi là *hàm nội tuyến* (inline function). Việc sử dụng hàm loại này cung cấp khả năng như khung nhìn nhưng cho phép chúng ta sử dụng được các tham số và nhờ đó tính linh hoạt sẽ cao hơn.

```
CREATE FUNCTION tên_hàm ([danh_sách_tham_số])  
RETURNS TABLE  
AS  
RETURN (câu_lệnh_select)
```

Inline function

Cú pháp của hàm nội tuyến phải tuân theo các quy tắc sau:

- Kiểu trả về của hàm phải được chỉ định bởi mệnh đề RETURNS TABLE.
- Trong phần thân của hàm chỉ có duy nhất một câu lệnh RETURN xác định giá trị trả về của hàm thông qua duy nhất một câu lệnh SELECT. Ngoài ra, không sử dụng bất kỳ câu lệnh nào khác trong phần thân của hàm.

Table-value Functions



```
CREATE FUNCTION func_XemSV(@khoa SMALLINT)
RETURNS TABLE
AS
RETURN(
    SELECT masv,hodem,ten,ngaysinh
    FROM sinhvien INNER JOIN lop
    ON sinhvien.malop=lop.malop
    WHERE khoa=@khoa
)
```

Với hàm được định nghĩa như trên, để biết danh sách các sinh viên khoá 25, ta sử dụng câu lệnh như sau:

```
SELECT * FROM dbo.func_XemSV(25)
```

còn câu lệnh dưới đây cho ta biết được danh sách sinh viên khoá 26

```
SELECT * FROM dbo.func_XemSV(26)
```

Table-value Functions



- Đối với hàm nội tuyến, phần thân của hàm chỉ cho phép sự xuất hiện duy nhất của câu lệnh RETURN. Trong trường hợp cần phải sử dụng đến nhiều câu lệnh trong phần thân của hàm, ta sử dụng cú pháp như sau để định nghĩa hàm:

```
CREATE FUNCTION tên_hàm([danh_sách_tham_số])  
RETURNS @biến_bảng TABLE định_nghĩa_bảng  
AS  
BEGIN  
    <các_câu_lệnh_trong_thân_hàm>  
RETURN  
END
```

multi-statement function

Table-value Functions



- Hàm xuất danh sách các khách hàng và tổng hóa đơn của khách hàng đó

```
CREATE FUNCTION ufn_SumOrderByCustomer(@CustomerId INT)
RETURNS @ResultTable TABLE
(
    CustomerId INT,
    CustomerFullName nvarchar(80),
    NumberOfOrder INT
)
AS
BEGIN
    INSERT INTO @ResultTable
    SELECT C.Id, CustomerFullName = C.FirstName + SPACE(1) + C.LastName, NumberOfOrder = COUNT(O.OrderNumber)
    FROM Customer C
    INNER JOIN [Order] O ON C.Id = O.CustomerId
    GROUP BY C.Id, C.FirstName, C.LastName
    HAVING C.Id = @CustomerId OR @CustomerId = 0

    RETURN
END
```

```
SELECT * FROM ufn_SumOrderByCustomer(0)
SELECT * FROM ufn_SumOrderByCustomer(85)
```

Results			
CustomerId	CustomerFullName	NumberOfOrder	
1	Maria Anders	6	
2	Ana Trujillo	4	
3	Antonio Moreno	7	
4	Thomas Hardy	13	
5	Christina Berglund	18	

Query executed successfully.

Results			
CustomerId	CustomerFullName	NumberOfOrder	
1	Paul Henriot	5	

Ví dụ



- Tạo một hàm trả về dữ liệu dạng bảng tùy theo giá trị của biến @CategoryID truyền vào:

```
CREATE FUNCTION fuGetProducts ( @CategoryID int )  
RETURNS TABLE  
AS  
RETURN  
(  
    SELECT  Categories.CategoryID, Categories.CategoryName, Products.ProductName,  
            Products.QuantityPerUnit, Products.UnitPrice  
    FROM    Categories  
    INNER JOIN Products ON Categories.CategoryID = Products.CategoryID  
    WHERE Categories.CategoryID=@CategoryID  
)
```

```
SELECT CategoryID, CategoryName, ProductName, QuantityPerUnit, UnitPrice  
FROM  dbo.fuGetProducts(1)
```


Ví dụ



```
--Table Valued Functions Examples [Multi-Statement]
CREATE FUNCTION CustomerPurchasedProductDetailsMultiStatement
(
    @p_CustomerID NVARCHAR(10)
)
RETURNS @CustomerPurchasedProducts TABLE(
    ProductName NVARCHAR(50),
    UnitPrice DECIMAL(8,2),
    AvailableStock INT)
AS
BEGIN
    © SQLServerCurry.com
    INSERT @CustomerPurchasedProducts
    SELECT P.ProductName,P.UnitPrice,P.UnitsInStock FROM
    Customers C INNER JOIN Orders O
    ON C.CustomerID=O.CustomerID INNER JOIN [Order Details] OD
    ON O.OrderID=OD.OrderID INNER JOIN Products P
    ON OD.ProductID=P.ProductID
    WHERE C.CustomerID=@p_CustomerID
    RETURN
END

--Test the function
SELECT * FROM dbo.CustomerPurchasedProductDetailsMultiStatement('ANTON')
```

Ví dụ



```
--ví dụ 1
USE AdventureWorks
GO
CREATE FUNCTION dbo.fn_ProductInfoByModelID(@p INT) RETURNS TABLE
AS
RETURN
    SELECT P.ProductID, P.Name, P.ProductNumber
    FROM Production.Product P
    WHERE P.ProductModelID = @p
GO
-- gọi hàm trực tiếp
SELECT * FROM dbo.fn_ProductInfoByModelID(5)

--hoặc join với hàm
SELECT TH.*, P.Name
FROM Production.TransactionHistory TH
JOIN dbo.fn_ProductInfoByModelID(5) P ON TH.ProductID = P.ProductID
```

So sánh in-line và multi-statement function



- - Hàm in-line chỉ yêu cầu khai báo đơn giản, bạn có thể thay đổi cấu trúc của tập kết quả tùy ý (ví dụ thêm một cột vào tập kết quả). Ngược lại, với hàm kiểu multi-statement bạn cần khai báo trước cấu trúc của tập kết quả, và nếu bạn thay đổi cấu trúc này thì cũng cần phải sửa lại khai báo.
- - Tuy nhiên khác biệt quan trọng nhất giữa hai loại hàm trên là ở hiệu năng.
- - *Khi bạn JOIN với một hàm in-line, bộ Optimizer có thể truy nhập vào thông tin index và statistics của các bảng được sử dụng trong hàm khi lựa chọn phương án thực thi. Nó biết các bảng nào được sử dụng và vì chỉ có một lệnh được dùng trong hàm.*
- - *Ngược lại, với hàm multi-statement, bộ Optimizer “mù tịt” với những gì xảy ra bên trong hàm. Nó vẫn thực hiện tối ưu hóa cho từng lệnh bên trong hàm, nhưng không thể “xé nhỏ” nó ra để tối ưu chung cho cả câu lệnh.*

So sánh in-line và multi-statement function



- Ví dụ hàm inline

```
CREATE FUNCTION dbo.fn_ProductInfoByModelID(@p INT) RETURNS TABLE
AS
RETURN
    SELECT P.ProductID, P.Name, P.ProductNumber
    FROM Production.Product P
    WHERE P.ProductModelID = @p
GO
```

- Ví dụ hàm multi-statement

```
CREATE FUNCTION dbo.fn_ProductInfoByModelID_MSTV(@p INT)
RETURNS @t TABLE(ProductID INT, Name NVARCHAR(50), ProductNumber NVARCHAR(25))
AS
BEGIN
    INSERT INTO @t
    SELECT P.ProductID, P.Name, P.ProductNumber
    FROM Production.Product P
    WHERE P.ProductModelID = @p

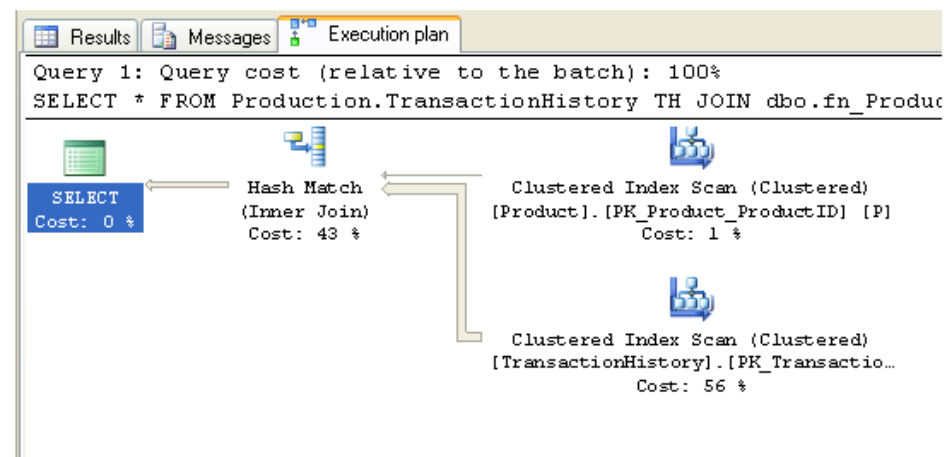
    RETURN
END
```

So sánh in-line và multi-statement function



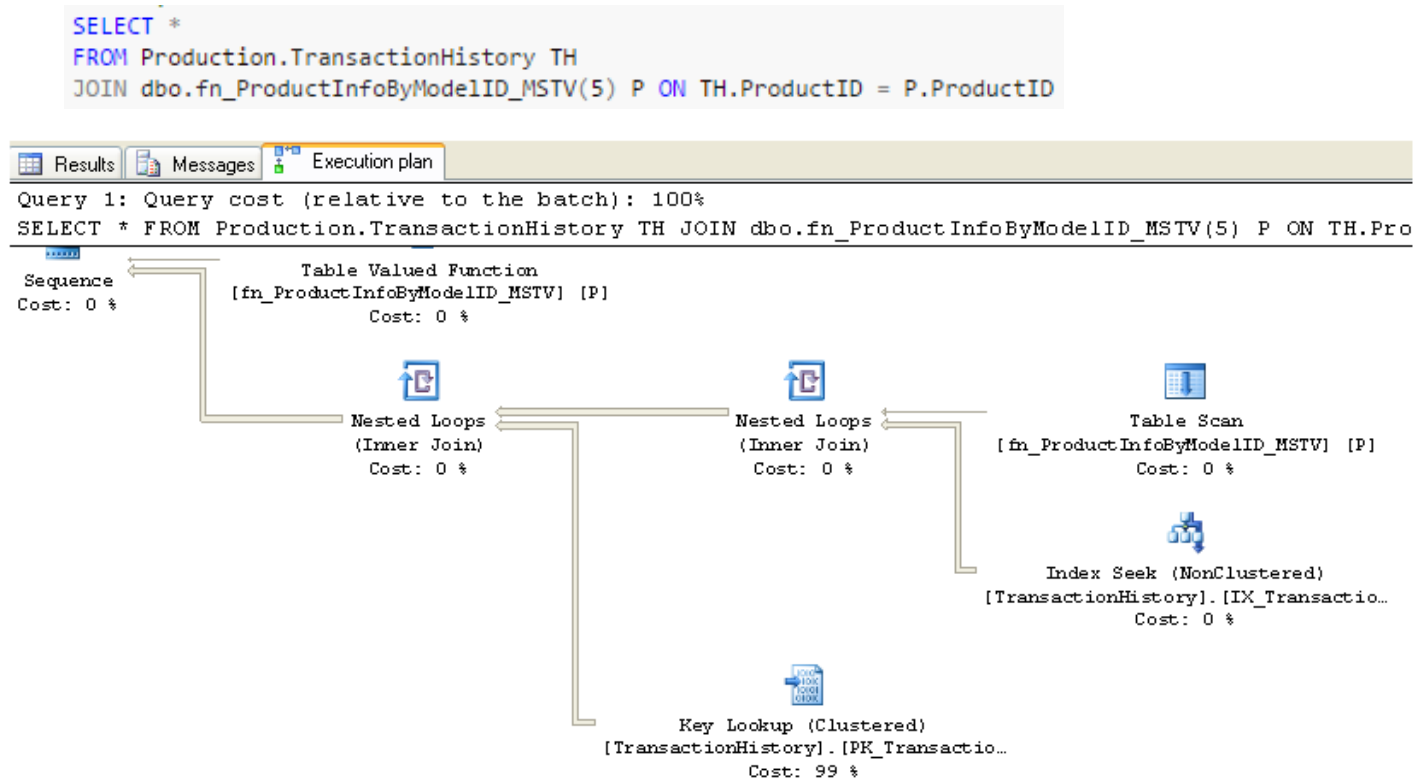
- Thực hiện truy vấn dùng in-line function

```
SELECT TH.*, P.Name  
FROM Production.TransactionHistory TH  
JOIN dbo.fn_ProductInfoByModelID(5) P ON TH.ProductID = P.ProductID
```



Bạn có thể thấy không có bước gọi đến hàm mà là hai bước *Clustered Index Scan* trên hai bảng và sau đó là [Hash Match](#), giống như câu lệnh *JOIN* trực tiếp giữa hai bảng

- Thực hiện truy vấn dùng multi-statement function



- Bây giờ bước thực hiện hàm đã trở thành một bước độc lập (trên cùng bên trái), và chi phí của nó được gán là 0% vì bộ Optimizer không có tí thông tin nào về bên trong của hàm. Khởi cần nói phương án thực thi giờ đã trở nên rườm rà như thế nào

- Và đây là thống kê vào/ra của hai lệnh gọi hàm:
- *“Khác biệt như vậy là khá đáng kể, số lần đọc (logical read) trên bảng TransactionHistory tăng từ 792 lên 6263, và số lần thực hiện (scan count) tăng từ 1 lên 10. Bạn có thể hỏi 10 scan count là từ đâu ra? Đó chính là số bản ghi mà hàm trả về.”*

Lệnh gọi hàm in-line (ví dụ 5):

```
Table 'Worktable'. Scan count 0, logical reads 0, physical reads 0, read-ahead reads 0, lob logical reads 0, lob physical reads 0, lob read-ahead reads 0.
```

```
Table 'TransactionHistory'. Scan count 1, logical reads 792, physical reads 8, read-ahead reads 788.
```

```
Table 'Product'. Scan count 1, logical reads 15, physical reads 3, read-ahead reads 14
```

Lệnh gọi hàm multi-statement (ví dụ 6):

```
Table 'TransactionHistory'. Scan count 10, logical reads 6263, physical reads 7, read-ahead reads 768.
```

```
Table '#0BC6C43E'. Scan count 1, logical reads 1, physical reads 0, read-ahead reads 0
```

Kết luận

- Bạn dùng hàm kiểu bảng khi dữ liệu trả về cần ở dạng bảng. Nó mềm dẻo hơn view vì có thể tiếp nhận tham số đầu vào và có thể chứa nhiều lệnh thao tác dữ liệu bên trong hàm (với kiểu multi-statement).
- Trong hai kiểu hàm in-line và multi-statement, trước hết bạn nên chọn viết kiểu in-line vì lý do khai báo đơn giản và có ưu thế về hiệu năng. Khi cần thiết phải dùng hàm multi-statement bạn nên khống chế số bản ghi trả về ở số lượng nhỏ.

SCHEMABINDING

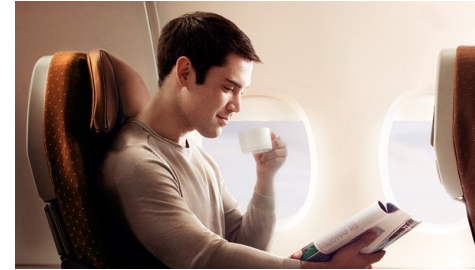


- Keeping things safe: functions can be schema-bound
 - Giúp cho các bảng cùng schema trong function giữ cấu trúc không thay đổi

```
CREATE FUNCTION Sales.CalculateSalesOrderTotal (@SalesOrderID INT)
RETURNS MONEY
WITH SCHEMABINDING AS
BEGIN
    DECLARE @SalesOrderTotal AS MONEY ;
    SELECT @SalesOrderTotal =
        SUM(sod.LineTotal)
        + soh.TaxAmt
        + soh.Freight
    FROM Sales.SalesOrderHeader AS soh
        INNER JOIN Sales.SalesOrderDetail AS sod
            ON soh.SalesOrderID = sod.SalesOrderID
    WHERE soh.SalesOrderID = @SalesOrderId
    GROUP BY soh.TaxAmt, soh.Freight ;
    RETURN @SalesOrderTotal ;
END;
GO
```


Dùng cho ràng buộc CHECK

- Giả sử chúng ta có rule như sau :
- *“no employee may have a salary that is 10 times greater than the salary of the lowest paid employee”*



```
CREATE FUNCTION dbo.SalaryWithinBounds ( @Salary MONEY )
RETURNS BIT
AS
BEGIN
    DECLARE @r_val AS BIT ;
    DECLARE @MinSalary AS MONEY ;

    SELECT @MinSalary = MIN(BaseSalary)
    FROM    dbo.Salaries

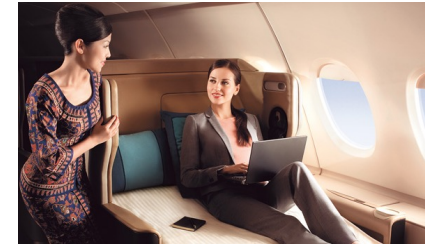
    IF ( @MinSalary * 10 ) > @Salary
        SET @r_val = 1
    ELSE
        SET @r_val = 0

    RETURN @r_val ;
END
GO

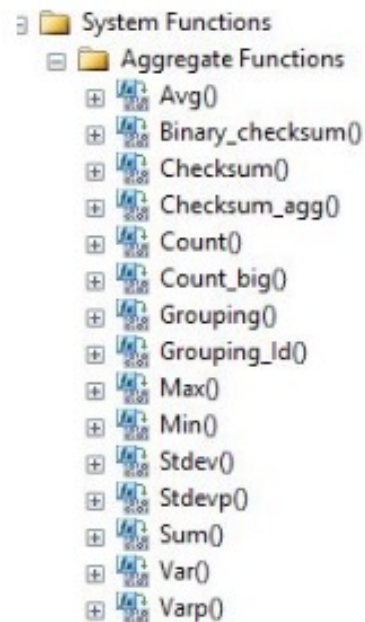
ALTER TABLE dbo.Salaries
ADD CONSTRAINT CheckMaxSalary
CHECK (dbo.SalaryWithinBounds(BaseSalary) = 1) ;
GO

/* This insert succeeds */
INSERT INTO dbo.Salaries
( EmployeeID, BaseSalary, Bonus )
VALUES ( 5, 1000, 0 ) ;
/* This insert will fail */
INSERT INTO dbo.Salaries
( EmployeeID, BaseSalary, Bonus )
VALUES ( 6, 1000000000, 50000 ) ;
```

Aggregate Functions in SQL Server



- SQL Server contains the following aggregates functions:



First create a table as in the following:

```
01. Create Table Student
02. (
03.   IId int Not Null primary key,
04.   Name Nvarchar(MAX) Not Null,
05.   Age Int Not Null,
06.   Class int not Null
07. )
```

Now, insert some values into the table.

```
01. Insert Into Student
02. Select 1,'A',12,10 Union All
03. Select 2,'B',16,11 Union All
04. Select 3,'C',15,9 Union All
05. Select 4,'D',13,12 Union All
06. Select 5,'E',14,11 Union All
07. Select 6,'F',17,8 Union All
08. Select 7,'G',12,7 Union All
09. Select 8,'H',17,12
```

Results		Messages		
	IId	Name	Age	Class
1	1	A	12	10
2	2	B	16	11
3	3	C	15	9
4	4	D	13	12
5	5	E	14	11
6	6	F	17	8
7	7	G	12	7
8	8	H	18	12

Aggregate Functions in SQL Server



• COUNT_BIG

- COUNT_BIG function returns the number of items in a group. COUNT_BIG works like the COUNT function. The only difference between the two functions is their return values. COUNT_BIG always returns a bigint data type value. COUNT always returns an int data type value.*

```
01. SELECT COUNT_BIG(age) AS Total , class AS Class
02. FROM dbo.student s
03. GROUP BY Class
```

Output

	Total	Class
1	1	7
2	1	8
3	1	9
4	1	10
5	2	11
6	2	12

CHECKSUM_AGG

- The CHECKSUM_AGG function returns the checksum of the values in a group. Null values are ignored. CHECKSUM_AGG can be used to detect changes in a table*

```
01. SELECT CHECKSUM_AGG(CAST(AGE AS int)) AS CHACKSUM
02. FROM student
```

Results	Messages
CHACKSUM	
1	31

```
01. UPDATE dbo.student
02. SET
03. AGE=28
04. WHERE IID=2;
05. SELECT CHECKSUM_AGG(CAST(AGE AS int)) AS CHACKSUM
06. FROM student
```

Results	Messages
CHACKSUM	
1	19

Aggregate Functions in SQL Server



- **CHECKSUM_AGG**
- *We can see that the value of CHECKSUM_AGG is changed that indicates some modification is performed into the table.*

```
01. SELECT IID,AGE,NAME,CLASS, CHECKSUM_AGG(CAST(AGE AS int)) AS CHACKSUM
02. FROM STUDENT GROUP BY IID,AGE,NAME,CLASS
```

Output

	IID	AGE	NAME	CLASS	CHACKSUM
1	1	12	A	10	12
2	2	16	B	11	16
3	3	15	C	9	15
4	4	13	D	12	13
5	5	14	E	11	14
6	6	17	F	8	17
7	7	12	G	7	12
8	8	17	H	12	17

Now we will update the table and calculate the checksum again.

```
01. UPDATE dbo.student
02. SET
03. AGE=22
04. WHERE IID%2=0;
05. SELECT IID,AGE,NAME,CLASS, CHECKSUM_AGG(CAST(AGE AS INT
06. FROM STUDENT GROUP BY IID,AGE,NAME,CLASS
```

Output

	IID	AGE	NAME	CLASS	CHACKSUM
1	1	12	A	10	12
2	2	22	B	11	22
3	3	15	C	9	15
4	4	22	D	12	22
5	5	14	E	11	14
6	6	22	F	8	22
7	7	12	G	7	12
8	8	22	H	12	22

Aggregate Functions in SQL Server



- **GROUPING**

- The GROUPING function indicates whether a specified column expression in a GROUP BY list is aggregated or not.

Syntax

GROUPING (column_name)

Arguments

column_name

Is a column in a GROUP BY clause to check for CUBE or ROLLUP null values.

Return type int

```
01. SELECT CLASS, SUM(AGE) AS [SUM] , GROUPING(CLASS) AS [GROUPING]
02. FROM STUDENT GROUP BY CLASS WITH ROLLUP
```

Output

	CLASS	SUM	GROUPING
1	7	12	0
2	8	17	0
3	9	15	0
4	10	12	0
5	11	30	0
6	12	30	0
7	NULL	116	1

Aggregate Functions in SQL Server



- **STDEV**

- The STDEV function returns the statistical standard deviation of all the values in the specified expression.

```
01. SELECT STDEV(AGE) AS STDEV_AGE , STDEV(CLASS) AS STDEV_CLASS FROM dbo.Student s;
```

Output

	STDEV_AGE	STDEV_CLASS
1	2.07019667802706	1.8516401995451

- **VAR**

- The VAR function returns the statistical variance of all values in the specified expression.

```
01. SELECT VAR(AGE) VAR_AGE, VAR(CLASS) VAR_CLASS FROM dbo.Student s;
```

Output

	VAR_AGE	VAR_CLASS
1	4.28571428571429	3.42857142857143

Aggregate Functions in SQL Server



• CHECKSUM

- The *CHECKSUM* function returns the checksum value computed over a row of a table, or over a list of expressions. *CHECKSUM* is intended for use in building hash indexes

```
01. SELECT * , CHECKSUM(*) as [CHECKSUM] FROM dbo.Student s
```

Output

Results		Messages			
	IId	Name	Age	Class	CHECKSUM
1	1	A	12	10	25290
2	2	B	16	11	22539
3	3	C	15	9	19193
4	4	D	13	12	10972
5	5	E	14	11	491
6	6	F	17	8	12824
7	7	G	12	7	9671
8	8	H	17	12	56604

```
SELECT * , CHECKSUM(*) as [CHECKSUM] FROM dbo.Student s  
  
UPDATE dbo.student  
SET AGE=30  
WHERE IID=2;  
  
SELECT * , CHECKSUM(*) as [CHECKSUM] FROM dbo.Student s
```

100 % ▾

Results

Messages

	IId	Name	Age	Class	CHECKSUM
1	1	A	12	10	25290
2	2	B	16	11	22539
3	3	C	15	9	19193
4	4	D	13	12	10972
5	5	E	14	11	491
6	6	F	17	8	12824
7	7	G	12	7	9671
8	8	H	17	12	56604

	IId	Name	Age	Class	CHECKSUM
1	1	A	12	10	25290
2	2	B	30	11	22763
3	3	C	15	9	19193
4	4	D	13	12	10972
5	5	E	14	11	491
6	6	F	17	8	12824
7	7	G	12	7	9671
8	8	H	17	12	56604

Configuration Functions



- *SET DATEFIRST specifies the first day of the week*

```
SET LANGUAGE Italian;
GO
SELECT @@DATEFIRST AS 'First Day'
      ,DATEPART(dw, SYSDATETIME()) AS 'Today';
GO
SET LANGUAGE us_english;
GO
SELECT @@DATEFIRST AS 'First Day'
      ,DATEPART(dw, SYSDATETIME()) AS 'Today';
GO
SET DATEFIRST 2;
GO
SELECT @@DATEFIRST AS 'First Day'
      ,DATEPART(dw, SYSDATETIME()) AS 'Today';
```

	First Day	Today
1	1	5

	First Day	Today
1	7	6

	First Day	Today
1	2	4

- System Functions
 - Aggregate Functions
 - Configuration Functions
 - ConnectionProperty()
 - @@Datefirst
 - @@Dbts
 - @@Langid
 - @@Language
 - @@Lock_Timeout
 - @@Max_Connections
 - @@Max_Precision
 - @@Nestlevel
 - @@Options
 - @@Remserver
 - @@Servername
 - @@Servicename
 - @@Spid
 - @@Textsize
 - @@Version

Configuration Functions



- *@@LANGID: Returns the local language identifier (ID) of the language that is currently being used.*

```
SET LANGUAGE 'Italian'  
SELECT @@LANGID AS 'Language ID'
```

Results		Messages
Language ID		
1	6	

- *@@LANGUAGE: Returns the name of the language currently being used.*

```
SET LANGUAGE 'us_english'  
SELECT @@LANGUAGE AS 'Language Name';
```

Results		Messages
Language Name		
1	us_english	

```
SELECT @@MAX_CONNECTIONS AS 'Max Connections';
```

Results		Messages
Max Connections		
1	32767	

- *@@MAX_CONNECTIONS: Returns the maximum number of simultaneous user connections allowed on an instance of SQL Server.*

Configuration Functions



- **@@MAX_PRECISION:** Returns the precision level used by **decimal** and **numeric** data types as currently set in the server

```
SELECT @@MAX_PRECISION AS 'Max Precision'
```

Results		Messages
Max Precision		
1	38	

- **@@NESTLEVEL:** Returns the nesting level of the current stored procedure execution (initially 0) on the local server

```
CREATE PROCEDURE usp_InnerProc AS
    SELECT @@NESTLEVEL AS 'Inner Level';
GO
CREATE PROCEDURE usp_OuterProc AS
    SELECT @@NESTLEVEL AS 'Outer Level';
    EXEC usp_InnerProc;
GO
EXECUTE usp_OuterProc;
GO
```

Results		Messages
Outer Level		
1	1	
Inner Level		
1	2	

Configuration Functions



- *@@SERVERNAME: Returns the name of the local server that is running SQL Server.*
- *@@SERVICENAME: Returns the name of the registry key under which SQL Server is running*

```
SELECT @@SERVERNAME AS 'Server Name'  
SELECT @@SERVICENAME AS 'Service Name';
```

Results		Messages
Server Name		
1	DMSPRO-ANHNT	
Service Name		
1	MSSQLSERVER	

- *@@SPID: Returns the session ID of the current user process.*

```
SELECT @@SPID AS 'ID', SYSTEM_USER AS 'Login Name', USER AS 'User Name';
```

Results		Messages
ID	Login Name	User Name
1	54	DMSpro-AnhNT\TuanTA
		dbo

Configuration Functions



- *@@TEXTSIZE: Returns the current value of the TEXTSIZE option.*

```
SELECT @@TEXTSIZE AS 'Text Size'
```

Results		Messages	
		Text Size	
1		2147483647	

- *@@VERSION: Returns system and build information for the current installation*

```
SELECT @@VERSION AS 'SQL Server Version';
```

Results		Messages	
		SQL Server Version	
1		Microsoft SQL Server 2014 - 12.0.2000.8 (X64) Feb 20 2014 20:04:26 Copyright (c) Microsoft Corporation Standard Edition (64-bit) on Windows NT 6.3 <X64> (Build 9600:)	

System Statistical Functions



- *@@CONNECTIONS*: Returns the number of attempted connections, either successful or unsuccessful since SQL Server was last started.

```
SELECT GETDATE() AS 'Today's Date and Time',  
       @@CONNECTIONS AS 'Login Attempts';
```

100 %		
Results Messages		
	Today's Date and Time	Login Attempts
1	2017-04-14 15:42:24.737	29514

- *@@CPU_BUSY*: Returns the time that SQL Server has spent working since it was last started. Result is in CPU time increments, or "ticks," and is cumulative for all CPUs, so it may exceed the actual elapsed time. Multiply by @@TIMETICKS to convert to microseconds.

```
SELECT @@CPU_BUSY * CAST(@@TIMETICKS AS float) AS 'CPU microseconds',  
       GETDATE() AS 'As of' ;
```

Results Messages		
	CPU microseconds	As of
1	145875000	2017-04-14 15:44:28.697

System Statistical Functions	
+	@@Connections
+	@@Cpu_Busy
+	::fn_Virtualestats()
+	@@Idle
+	@@Io_Busy
+	@@Pack_Received
+	@@Pack_Sent
+	@@Packet_Errors
+	@@Timeticks
+	@@Total_Errors
+	@@Total_Read
+	@@Total_Write

System Statistical Functions



- *@@IDLE : Returns the time that SQL Server has been idle since it was last started. The result is in CPU time increments, or "ticks," and is cumulative for all CPUs, so it may exceed the actual elapsed time. Multiply by @@TIMETICKS to convert to microseconds.*

```
SELECT @@IDLE * CAST(@@TIMETICKS AS float) AS 'Idle microseconds',  
        GETDATE() AS 'as of';
```

Results		Messages
	Idle microseconds	as of
1	2451057468750	2017-04-14 15:49:07.653

- *@@IO_BUSY: Returns the time that SQL Server has spent performing input and output operations since SQL Server was last started.*

```
SELECT @@IO_BUSY*@@TIMETICKS AS 'IO microseconds',  
        GETDATE() AS 'as of';
```

Results		Messages
	IO microseconds	as of
1	46437500	2017-04-14 15:50:35.720

System Statistical Functions



- @@PACKET_ERRORS : Returns the number of network packet errors that have occurred on SQL Server connections since SQL Server was last started
- @@PACKET_RECEIVED:Returns the number of input packets read from the network by SQL Server since it was last started.
- @@PACKET_SENT:Returns the number of output packets written to the network by SQL Server since it was last started.

```
SELECT @@PACKET_ERRORS AS 'Packet Errors';  
SELECT @@PACK_RECEIVED AS 'Packets Received';  
SELECT @@PACK_SENT AS 'Pack Sent';
```

Results		Messages
Packet Errors		
1	0	
Packets Received		
1	92620	
Pack Sent		
1	27962	

System Statistical Functions



- @@TOTAL_READ : Returns the number of disk reads, not cache reads, by SQL Server since SQL Server was last started.
- @@TOTAL_WRITE: Returns the number of disk writes by SQL Server since SQL Server was last started.
- @@TOTAL_ERRORS : Returns the number of disk write errors encountered by SQL Server since SQL Server last started.

```
SELECT @@TOTAL_READ AS 'Reads', @@TOTAL_WRITE AS 'Writes', GETDATE() AS 'As of';
```

100 %			
Results Messages			
	Reads	Writes	As of
1	8920	333180	2017-04-14 15:54:07.110

100 %			
Results Messages			
	Reads	Writes	As of
1	8920	333180	2017-04-14 15:54:07.110

Một số hàm hữu ích



```
CREATE FUNCTION dbo.fnCSVStr2Table(@CSVStr VARCHAR(8000),@Delimiter VARCHAR(1))
RETURNS @Tbl TABLE (ValueColumn VARCHAR(1000))
AS
BEGIN
    DECLARE @SubStr VARCHAR(100), @i INT
    SET @i = CHARINDEX(@Delimiter, @CSVStr, 0)
    WHILE @i > 0
    BEGIN
        SET @SubStr = LEFT(@CSVStr,@i-1)
        INSERT INTO @Tbl SELECT @SubStr
        SET @CSVStr = SUBSTRING(@CSVStr, @i+1,8000)
        SET @i = CHARINDEX(@Delimiter, @CSVStr, 0)
    END

    INSERT INTO @Tbl
    SELECT LTRIM(RTRIM(@CSVStr))
RETURN
END
```

Hàm chuyển chuỗi thành bảng

```
SELECT * FROM dbo.fnCSVStr2Table('A,B,C,D,E,F', ',');
SELECT * FROM dbo.fnCSVStr2Table('A;B;C;D;E;F', ';');
```

Results		Messages
ValueColumn		
1	A	
2	B	
3	C	
4	D	
5	E	
6	F	

Một số hàm hữu ích



```
CREATE FUNCTION [dbo].[ufn_ConvertNumbertoString]
(
    @Number int,
    @Size    int
)
RETURNS varchar(10)
AS
BEGIN
```

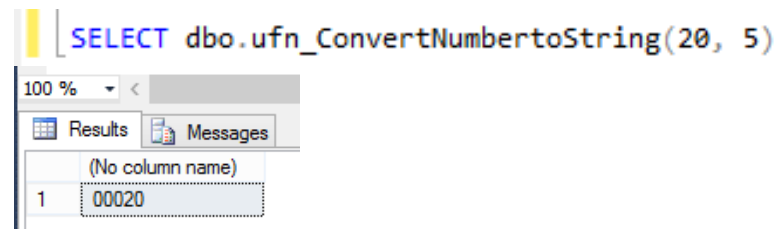
```
    DECLARE @OutputString VARCHAR(10);
```

```
    SET @OutputString = REPLICATE('0',@Size-LEN(RTRIM(CONVERT(varchar(8000),@Number))))
    + CONVERT(varchar(8000),@Number)
```

```
    -- Return the result of the function
    RETURN @OutputString
```

```
END
```

Hàm chuyển số thành chuỗi có padding số 0




```

WHILE (@COUNTER <=LEN(@strInput))
BEGIN
    SET @COUNTER1 = 1
    --Tìm trong chuỗi mẫu
    WHILE (@COUNTER1 <=LEN(@SIGN_CHARS)+1)
    BEGIN
        IF UNICODE(SUBSTRING(@SIGN_CHARS, @COUNTER1,1))
            = UNICODE(SUBSTRING(@strInput,@COUNTER ,1) )
        BEGIN
            IF @COUNTER=1
                SET @strInput = SUBSTRING(@UNSIGN_CHARS, @COUNTER1,1)
                + SUBSTRING(@strInput, @COUNTER+1,LEN(@strInput)-1)
            ELSE
                SET @strInput = SUBSTRING(@strInput, 1, @COUNTER-1)
                +SUBSTRING(@UNSIGN_CHARS, @COUNTER1,1)
                + SUBSTRING(@strInput, @COUNTER+1,LEN(@strInput)- @COUNTER)
                BREAK
            END
            SET @COUNTER1 = @COUNTER1 +1
        END
    --Tìm tiếp
    SET @COUNTER = @COUNTER +1
    END
    SET @strInput = replace(@strInput,',',' ')
    RETURN @strInput
END

```

```

DECLARE @Str NVARCHAR(30)
SET @Str = N'Chúc một ngày vui vẻ';
select @Str AS 'Tiếng Việt Có Dấu',
       dbo.ufn_ConvertToBareVNLang(@Str) AS 'Tieng Viet Khong Dau'

```

Results		Messages
Tiếng Việt Có Dấu		Tieng Viet Khong Dau
1	Chúc một ngày vui vẻ	Chuc mot ngay vui ve

Một số hàm hữu ích



- Hàm cho phép lấy thời điểm tạo database

```
CREATE FUNCTION dbo.DBCreationDate
( @dbname sysname )
RETURNS datetime
AS
BEGIN
    DECLARE @crdate datetime
    SELECT @crdate = crdate FROM master.dbo.sysdatabases
        WHERE name = @dbname
    RETURN ( @crdate )
END
GO
```

```
SELECT dbo.DBCreationDate('Northwind')
GO
```

Results		Messages
(No column name)		
1	2017-03-06 10:09:37.930	

Một số hàm hữu ích



- Hàm cho phép tính số ngày làm việc (working day) giữa hai ngày

```
CREATE FUNCTION dbo.GetWorkingDays
( @StartDate datetime,
  @EndDate datetime )
RETURNS INT
AS
BEGIN
  DECLARE @WorkDays int, @FirstPart int
  DECLARE @FirstNum int, @TotalDays int
  DECLARE @LastNum int, @LastPart int
  IF (DATEDIFF(day, @StartDate, @EndDate) < 2)
  BEGIN
    RETURN ( 0 )
  END
```

Một số hàm hữu ích



```
SELECT
  @TotalDays = DATEDIFF(day, @StartDate, @EndDate) - 1,
  @FirstPart = CASE DATENAME(weekday, @StartDate)
    WHEN 'Sunday' THEN 6
    WHEN 'Monday' THEN 5
    WHEN 'Tuesday' THEN 4
    WHEN 'Wednesday' THEN 3
    WHEN 'Thursday' THEN 2
    WHEN 'Friday' THEN 1
    WHEN 'Saturday' THEN 0
  END,
  @FirstNum = CASE DATENAME(weekday, @StartDate)
    WHEN 'Sunday' THEN 5
    WHEN 'Monday' THEN 4
    WHEN 'Tuesday' THEN 3
    WHEN 'Wednesday' THEN 2
    WHEN 'Thursday' THEN 1
    WHEN 'Friday' THEN 0
    WHEN 'Saturday' THEN 0
  END
```

Một số hàm hữu ích



```
IF (@TotalDays < @FirstPart)
BEGIN
    SELECT @WorkDays = @TotalDays
END
ELSE
BEGIN
    SELECT @WorkDays = (@TotalDays - @FirstPart) / 7
    SELECT @LastPart = (@TotalDays - @FirstPart) % 7
    SELECT @LastNum = CASE
        WHEN (@LastPart < 7) AND (@LastPart > 0) THEN @LastPart - 1
        ELSE 0
    END
    SELECT @WorkDays = @WorkDays * 5 + @FirstNum + @LastNum
END
RETURN ( @WorkDays )
END
GO
```

```
SELECT dbo.GetWorkingDays ('03/01/2017', '03/31/2017')
GO
```

(No column name)	
1	21



THANK YOU

Dr. Tran Anh Tuan, Faculty of Mathematics and Computer Science, University of
Science, HCMC